A Privacy-Preserving Selectively Disclosed eKYC System Using Merkle Tree and Zero-Knowledge Proofs

Istiaque Ahmed
Graduate School of Informatics
Osaka Metropolitan University (OMU)
Osaka, Japan
sw23837u@st.omu.ac.jp

Tadashi Nakano
Graduate School of Informatics
Osaka Metropolitan University (OMU)
Osaka, Japan
tnakano@omu.ac.jp

Kentaroh Toyoda
Institute of
High Performance Computing (IHPC)
*A*STAR*, Singapore
kentaroh.toyoda@ieee.org

Thi Hong Tran
Graduate School of Informatics
Osaka Metropolitan University (OMU)
Osaka, Japan
hong@omu.ac.jp

***Abstract*—Digital identity verification has become crucial to every service in daily life. The privacy concerns associated with traditional Know Your Customer (KYC) systems have come to the forefront. These systems often require the sharing of personal information, which is stored in centralized databases, making them vulnerable to unauthorized access. To address these challenges, this work implements an electronic KYC system with selective disclosure using Merkle Tree and Zero-Knowledge Proofs (ZKP). Selective disclosure enables users to share only the necessary information, thereby reducing the exposure of sensitive data. ZKP enables the verification of this information without revealing the actual data, ensuring that privacy is preserved. The combination of selective disclosure and zkSNARKs in the proposed framework provides a solution for generating a single proof compared to multiple market proofs. This work demonstrates significant improvements in privacy protection compared to traditional identification systems. The implementation process and performance evaluation explore its potential impact on eKYC.**

***Index Terms*—electronic Know Your Customer (eKYC), Decentralized Identity (DID), Self-Sovereign Identity (SSI), Zero-knowledge proofs (ZKP), Blockchain**

I. Introduction

Financial services are rapidly digitizing, resulting in substantial breakthroughs in consumers' identity verification. eKYC [1]–[3] solutions are becoming the norm for onboarding new customers. These systems offer numerous advantages, including increased efficiency, reduced costs, and improved regulatory compliance. However, it also presents complex challenges, particularly concerning the security and protection of personal data. As financial institutions increasingly rely on eKYC systems, there is an urgent need to address these challenges and develop solutions that protect user privacy without compromising regulatory compliance.

Traditional Know Your Customer (KYC) procedures involved manual verification of documents and face-to-face interactions, which often resulted in significant delays. In contrast, eKYC [1], [2] offers digital platforms that enable real-time identity verification using data from government databases, biometrics, and other digital identifiers. With the evolution of eKYC, financial institutions are better equipped to meet identity verification demands in the digital economy. eKYC is a process used to verify the identity of customers remotely through online or electronic means. It simplifies the customer onboarding process by replacing conventional paper-based identification methods. This process includes the collection and verification of identity documents, biometrics such as facial recognition or fingerprinting, and other personal information via

In eKYC, selective disclosure allows institutions to verify a customer with a minimum amount of personal information. For example, instead of providing a full address or date of birth, a user might only need to confirm their age or residential status. This approach reduces the risk of data breaches and ensures compliance with privacy regulations. Selective disclosure uses ZKPs [4], [5] to prove specific identity attributes without revealing private data. ZKPs are crucial for privacy, allowing users to prove necessary facts without revealing their birthdate. In this article, we utilize zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) [6], [7] for both the prover and the verifier. The proof generation in zk-SNARKs is mathematically described as follows.

$$\pi = \text{Prove}(x, w)$$

Where x is the public input and w is the private witness. The prover uses these inputs to generate a succinct, non-interactive proof π.

Proof verification is carried out by the verifier using the public input x and the proof π to check its validity without requiring access to the private witness w. The verification condition is:

$$\text{Verify}(x, \pi) = \begin{cases} \text{True} & \text{if } \pi \text{ is valid} \\ \text{False} & \text{otherwise} \end{cases}$$

The critical properties of zk-SNARKs are *succinctness* and *non-interactivity*, ensuring that the proof π is compact and

easily verifiable, making it ideal for use in scalable eKYC systems.

In this article, we use Groth16 zk-SNARKs [6], [8], [9], a cryptographic protocol that enables someone to prove knowledge of specific information such as personal data without revealing that information. The process begins by representing the computation as a set of linear equations in a Rank-1 Constraint System (R1CS), expressed as follows.

$$A \cdot Z = B \cdot Z = C \cdot Z$$

Where Z represents the variables, and A, B, and C are vectors defining the constraints. These equations are then transformed into a polynomial equation within a Quadratic Arithmetic Program (QAP), where the polynomial $P(x)$ is formed as below.

$$P(x) = \left(\sum A_i(x) \cdot Z_i\right) \cdot \left(\sum B_i(x) \cdot Z_i\right) - \left(\sum C_i(x) \cdot Z_i\right)$$

This polynomial must be divisible by a target polynomial $t(x)$, such that $P(x) = h(x) \cdot t(x)$, where $h(x)$ is the quotient polynomial. To ensure zero-knowledge and preserve privacy, the prover adds randomness to the following polynomials.

$$A'(x) = A(x) + r_A \cdot t(x), \quad B'(x) = B(x) + r_B \cdot t(x)$$

Finally, the verifier checks the proof's validity by verifying a bilinear pairing equation:

$$e\left(A'(x)_1, B'(x)_1\right) = e\left(C'(x)_1, \text{generator}_2\right) \cdot e\left(\text{Proof}(h(x))_1, \text{generator}_2\right)$$

This ensures the verifier doesn't access actual data, verifying commitments and proof validity.

II. Problem Statement and Motivation

Identification systems typically require users to provide extensive personal data, which is then stored in centralized databases. Such centralized repositories [10] are attractive targets for cybercriminals, heightening the risk of data breaches that can lead to identity theft, financial loss, and reputational damage for institutions. Moreover, traditional KYC systems often request more information than is necessary for verification. For instance, users may be asked to submit their full date of birth or complete address when only proof of age or residency is required, as shown in Fig. 1. This overexposure of personal data not only increases the risk of misuse but also contradicts emerging data privacy regulations that emphasize user consent and data minimization. It is essential to comply with frameworks like the General Data Protection Regulation (GDPR) [11] in Europe and similar regulations in other jurisdictions. These laws mandate strict rules for the collection, storage, and processing of personal information. As a result, financial institutions must adopt more privacy-conscious identity verification mechanisms.

By integrating Merkle Trees and Zero-Knowledge Proofs (ZKPs), the proposed framework enhances privacy and security within eKYC processes. This approach not only safeguards sensitive user data but also ensures compliance with regulatory requirements, effectively balancing user expectations and legal obligations. We are highly motivated to generate a single proof of the disclosed set of attributes, opposing multiple Markel proofs. Additionally, incorporating SSI and DID principles [12], [13] further mitigates the risks associated with centralized data storage by empowering users to retain control over their personal information.

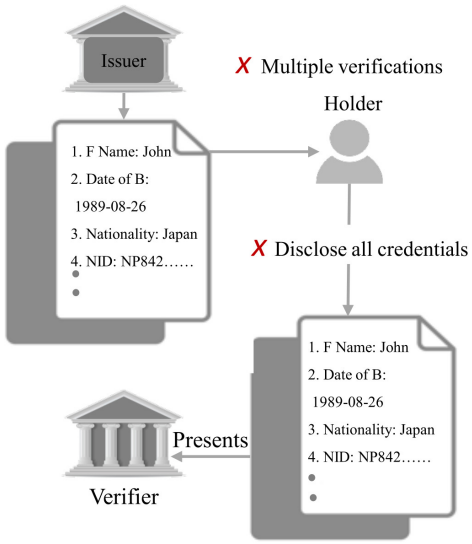

Fig. 1. Issues of existing KYC.

III. Literature Review

TABLE I
Comparison of Privacy-Preserving eKYC Solutions.

Research	Privacy	ZKP	Scalability	Selective Disclosure
Bhatia S et al. [2]	✓	×	✓	×
Chandraprabha K et al. [14]	✓	×	✓	×
Jaber B et al. [15]	✓	×	✓	×
Fugkeaw S et al. [16]	✓	×	✓	✓
Proposed Work	✓	✓	✓	✓

Blockchain-based eKYC [14], [15], [17] systems are designed to streamline the identity verification process by leveraging digital technologies. The article by Hannan *et al.* [17] is a systematic review article on blockchain-based eKYC. In this review, the authors focused on the application areas of eKYC, the platforms, and the technologies that were implemented. Several technologies and protocols are employed to ensure secure and reliable identity verification. Chandraprabha [14] and Jaber *et al.* [15] also proposed blockchain-based eKYC frameworks for the banking sector, addressing challenges such as regulatory compliance, data privacy, scalability, and costs.

Since biometric data is unique to each individual, it provides a highly secure method for identity verification. The proposed solution by Bhatia *et al.* [2] is a complete video KYC. Some eKYC systems use blockchain [18] to store and verify identity data in a decentralized, tamper-proof way, ensuring transparency and secure verification. Fugkeaw [16] proposed the eKYC TrustBlock system, which

utilizes Ciphertext-Policy Attribute-Based Encryption (CP-ABE) to secure sensitive data by binding encryption with specific attributes.

In eKYC systems [16], ZKPs [19] enable identity verification without revealing sensitive information. Li *et al.* [19] propose a privacy-preserving identity system for ridesharing using ZKP and blockchain, implemented on Hyperledger Fabric with the Ursa cryptographic library. Their results demonstrate strong suitability for real-world applications. Similarly, cryptocurrencies like Zcash use zk-SNARKs to ensure transaction privacy. Eddien *et al.* [20] analyze blockchain-based SSI systems, outlining challenges, proposing a layered architecture, and emphasizing interoperability and usability.

Based on the reviewed works, we compare our proposed solution with closely related studies in terms of privacy, ZKP usage, scalability, and selective disclosure. Table I summarizes the key differences and highlights our unique contributions.

IV. Proposed Privacy Preserving Approach

In this proposed work, we enhance privacy in the eKYC system by integrating selective disclosure and ZKPs. The process comprises several participants, including the *Identity Issuer*, the *Holder* with a *Wallet*, and the *Verifier*. The overall functional diagram of the proposed system is shown in Fig. 2.

The *Identity Issuer* is responsible for issuing digital identities to users after verifying their credentials through traditional or digital means, ensuring the authenticity of user identity attributes such as age, nationality, or address. In the proposed architecture, the government body serves as the identity provider. The *Holder Wallet* holds all the verifiable credentials, acting as a secure digital repository under the user's control, enabling them to manage and selectively disclose specific attributes. The wallet interacts with both the Identity Issuer and the Verifier. The *Holder* in the proposed architecture is the user. The *Verifier*, such as a financial institution, requests identity verification from the user and only receives the information necessary for a specific transaction. This process is facilitated through selective disclosure and zkSNARKs, where the selected information is submitted to the verifier in the form of a cryptographic proof.

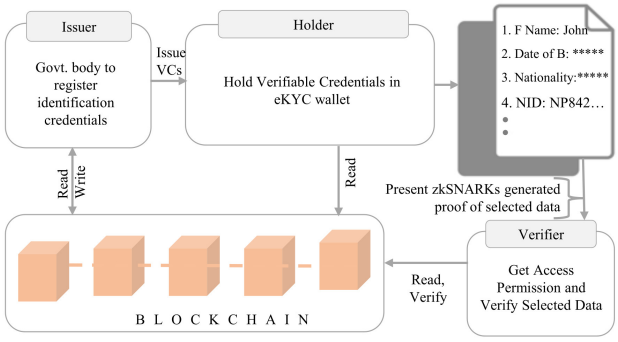

Fig. 2. Proposed eKYC Using Selective Disclosure and zkSNARKs.

To understand the proposed solution, consider a situation where a student has an ID card issued by the University. A club wants to verify the student using the credentials embedded in the student's ID card. Instead of sharing the entire ID, the student can use the system to prove only the required attributes, such as the student's name, University, age¿18, without revealing other personal data.

As described in Algorithm 1, the issuer (e.g., University) begins by preparing an attribute list $A = [a_0, a_1, \ldots, a_{n-1}]$, where each a_i represents a user-specific attribute such as name, country, or date of birth. To ensure privacy and uniqueness, a corresponding salt list $S = [s_0, s_1, \ldots, s_{n-1}]$ is generated, where each s_i is a random value used to obfuscate a_i.

The issuer computes a salted hash for each attribute using a cryptographic hash function $H(\cdot)$, such as the Poseidon hash: $h_i = H(a_i, s_i)$. These hashed values h_i form the leaf nodes of a Merkle Tree T, which is then constructed to compute the Merkle root $R = T.\text{root}$. The issuer digitally signs the root using its private signing key sk_{issuer} to produce a signature $\sigma = \text{Sign}(sk_{\text{issuer}}, R)$, which binds the data to the issuer's identity. For each attribute, the corresponding Merkle path $p_i = T.\text{getPath}(h_i)$ is also computed to allow future selective disclosure. The final output includes the Merkle root R, signature σ, the full attribute list A, salts S, and all Merkle paths $P = [p_0, p_1, \ldots, p_{n-1}]$, enabling the holder to later prove possession of specific attributes without revealing the entire dataset.

Algorithm 1 Merkle Tree Construction and Root Signing by Issuer.

Require: Attribute list $A = [a_0, a_1, \ldots, a_{n-1}]$
Salt list $S = [s_0, s_1, \ldots, s_{n-1}]$
Poseidon hash function $H(\text{value}, \text{salt})$
Issuer's private signing key sk_{issuer}
Ensure: Merkle Root R, Signature σ, Merkle Paths $P = [p_0, p_1, \ldots, p_{n-1}]$
Initialize empty list $L \leftarrow [\,]$
for $i = 0$ to $n - 1$ **do**
  Compute salted hash: $h_i \leftarrow H(a_i, s_i)$
  Append h_i to list L
end for
Construct a Merkle tree T from leaf list L
Extract Merkle root: $R \leftarrow T.\text{root}$
Sign the Merkle root: $\sigma \leftarrow \text{Sign}(sk_{\text{issuer}}, R)$
for $i = 0$ to $n - 1$ **do**
  Compute Merkle path: $p_i \leftarrow T.\text{getPath}(h_i)$
end for
return Merkle Root R, Signature σ, Merkle Paths $P = [p_0, p_1, \ldots, p_{n-1}]$, Attribute List A, Salt List S

The blockchain acts as a decentralized ledger, where the *Issuer* stores the root hash of the credentials issued to the *Holder*, ensuring that the credentials are immutable and tamper-proof. Rather than storing the full credentials directly on the blockchain, the Issuer constructs a Merkle Tree from the credential data and stores only the root hash of this tree on the blockchain. The Holder generates zkSNARKs proofs based on selected information. The blockchain then serves as a reference point where the *Verifier* can access the Merkle root, enabling the Verifier to confirm the authenticity of the Holder's data without accessing the full credential set.

As outlined in Algorithm 2, the holder generates a zk-SNARK proof to demonstrate that selected identity attributes are authentic and satisfy a given condition without re-

Algorithm 2 zkSNARK Proof Generation for Selective Attribute Disclosure.

Require: Private Inputs:
dobYear (e.g., 1989), salt
Merkle proof paths:
pathElements_dob[], pathIndices_dob[]
pathElements_name[], pathIndices_name[]
pathElements_uni[], pathIndices_uni[]
Require: Public Inputs:
merkleRoot, currentYear = 2025, minAge = 18
Poseidon("Ahmed"), Poseidon("Osaka Metropolitan University")
Ensure: zkSNARK proof attesting that:
age $\geq$ minAge,
name = "Ahmed",
university = "Osaka Metropolitan University"
dobLeaf $\leftarrow$ `Poseidon(dobYear, salt)`
nameLeaf $\leftarrow$ `Poseidon(`"$Ahmed$", `salt)`
uniLeaf $\leftarrow$ `Poseidon(`
"Osaka Metropolitan University", `salt)`
root_dob $\leftarrow$ `MerkleRoot(`
dobLeaf, pathElements_dob, pathIndices_dob)
root_name $\leftarrow$ `MerkleRoot(`
nameLeaf, pathElements_name, pathIndices_name)
root_uni $\leftarrow$ `MerkleRoot(`
uniLeaf, pathElements_uni, pathIndices_uni)
assert(root_dob == merkleRoot)
assert(root_name == merkleRoot)
assert(root_uni == merkleRoot)
age $\leftarrow$ currentYear $-$ dobYear
assert(age $\geq$ minAge)
return zkSNARK proof

vealing sensitive data. The process begins with private inputs, the holder's year of birth, name, and university attributes. The cryptographic salts, and Merkle proof paths (`pathElements[]`, `pathIndices[]`) are also the private inputs. Public inputs include the signed `merkleRoot`, current year (2025), and minimum age (18). Merkle proof paths verify inclusion in the original tree by checking matching roots. The holder calculates age using `age = currentYear - dobYear` and verifies the age requirement. If valid, a zkSNARK proof is generated and returned for the verifier.

When the Verifier needs to confirm the holder's identity, they access the zkSNARK proof and verify it against the root hash stored on the blockchain. The process allows the Verifier to efficiently confirm that the holder's selected data is part of the credentials stored. As described in Algorithm 3, a verifier such as a club or service provider validates the authenticity and correctness of the received zkSNARK proof and credential data. The process begins by verifying the digital signature σ on the Merkle root `merkleRoot` using the issuer's public key pk_{issuer}, ensuring that the root was genuinely issued by a trusted authority. If the signature is invalid, the verification fails immediately.

The verifier runs the Groth16 zkSNARK verifier to check the validity of the submitted proof (`proof.json`) against the public inputs (`public.json`) using the verification key (`verification_key.json`). The verifier then extracts the Merkle root from the public inputs (`publicMerkleRoot`) and checks that it matches the signed root to prevent substitution attacks. Internally, the zkSNARK circuit has already proven that the attributes "Ahmed" and "Osaka Metropolitan University" are valid Merkle leaves whose inclusion paths reconstruct the same `publicMerkleRoot`, and that the holder's age is computed from the hidden birth year meets the condition `currentYear - dobYear` $\geq$ 18. If all these checks succeed, the verifier accepts the proof and returns `true`; otherwise, the process halts and returns `false`.

Algorithm 3 Verifier Validates zkSNARK Proof and Credential Integrity.

Require: zkSNARK proof: `proof.json`
Public inputs: `public.json`
Verification key: `verification_key.json`
Signed Merkle root: `merkleRoot`
Digital signature: σ
Issuer's public key: pk_{issuer}
Poseidon hashes: `Poseidon("Ahmed")`, `Poseidon("Osaka Metropolitan University")`
Ensure: isValid $\in$ {`true`, `false`}
Step 1: Verify Digital Signature of Merkle Root.
isSigValid $\leftarrow$ `VerifySignature(`pk_{issuer}, `merkleRoot`, σ)
if isSigValid == `false` **then**
return `false`
end if
Step 2: Verify zkSNARK Proof
isProofValid $\leftarrow$ `snarkjs groth16 verify verification_key.json public.json proof.json`
if isProofValid == `false` **then**
return `false`
end if
Step 3: Check Merkle Root Consistency
publicMerkleRoot $\leftarrow$ `public.json[0]`
if publicMerkleRoot $\neq$ `merkleRoot` **then**
return `false`
end if
Step 4: (Inside zkSNARK Circuit – Already Proven)
– `Poseidon("Ahmed")` and `Poseidon("Osaka Metropolitan University")` are Merkle leaves
– Their Merkle paths reconstruct `publicMerkleRoot`
– Age condition: `currentYear - dobYear` $\geq$ 18
Step 5: Return Final Result
return `true`

V. Implementation and Results Discussion

The implementation used Remix IDE with ZoKrates plugin, and Remix VM as the test blockchain. Focus was on the Groth16 proving system for zkSNARK proof generation and verification. Python handled preprocessing, Solidity was used

TABLE II
SYSTEM PARAMETERS AND METRICS.

Metric	Value
Number of Total Attributes	6
Number of Disclosed Fields	3 (DOB, Name, University)
Merkle Tree Depth	3
zkSNARK Proof Size	1.5 KB
Verification Key Size	34 KB
Signature Scheme	EdDSA
Hash Function Used	Poseidon

for smart contracts, and Poseidon generated hashes at various stages.

Implementation of Proposed Work: We followed the standard ZoKrates workflow: *Compilation using Groth16*, *Compute*, *Setup*, *Generate Proof*, *Export Verifier*, and finally *On-Chain Verification*. The implementation codes are provided in the corresponding GitHub repository [21].

Compilation is performed using ZoKrates' Groth16, a zkSNARK protocol enabling efficient, privacy-preserving proofs. It is widely used due to its succinctness and non-interactive nature, allowing verification with a single proof. In this step, the ZoKrates toolchain compiles the Solidity code into an arithmetic circuit, representing program logic as mathematical constraints. The *Compute* step generates the *witness*, a valid assignment of variables within the circuit. This witness includes input values and intermediate values from executing the program, satisfying all circuit constraints and demonstrating input validity. The witness is essential for generating the zkSNARK proof in subsequent steps.

In the *Setup* step, a proving key and verification key are generated from a source of randomness, known as "toxic waste." The verification key is crucial for zkSNARK proof validation without revealing input data. The generated `verification_key.json` file includes elliptic curve parameters and other necessary cryptographic elements for on-chain validation. The *Generate Proof* step produces a cryptographic proof using the proving key and the computed witness. This proof demonstrates that the prover knows the input data satisfying the circuit's constraints without revealing it. The proof is stored in `proof.json` and can be verified using the Groth16 protocol.

To *Export Verifier*, the `verifier.sol` Solidity contract is deployed on the Ethereum blockchain to verify zkSNARK proofs using the `verification_key.json`. The Groth16 protocol ensures efficient verification with minimal elliptic curve operations. The contract contains a function that checks the proof against the verification key and returns `true` if valid, or `false` if not. This verifier enables trustless, on-chain verification of eKYC proofs. By deploying `verifier.sol`, zkSNARKs ensure verification as shown in Fig. 3.

Table II presents the system parameters and cryptographic configurations used in the proposed zkSNARK-based selective disclosure framework. The total number of attributes stored in the Merkle Tree is six, out of which three fields, Date of Birth (DOB), Name, and University, are selectively disclosed during proof generation. The Merkle Tree has a depth of 3, supporting up to 8 leaves, and the zkSNARK proof size is compact at 1.5 KB. The verification key, required for proof validation, is approximately 34 KB. The system employs EdDSA for digital signatures and Poseidon as the zk-friendly hash function.

TABLE III
PERFORMANCE TIMING SUMMARY.

Phase	Operation	Time (ms)
Merkle Tree Construction	Attribute Hashing + Tree Build	12.7
Merkle Root Signing	Root Signing (EdDSA)	2.1
Merkle Root Verification	Signature Verification (EdDSA)	1.8
Proof Generation	Total Generation Time	312.1
Verification	Total Verification Time	96.3

Table III summarizes the performance timing results of the entire protocol. Merkle Tree construction, including attribute hashing and tree building, takes 12.7 ms, while signing the Merkle root using EdDSA adds an additional 2.1 ms. zk-SNARK proof generation, which includes hash recomputation and full circuit execution, requires 312.1 ms, demonstrating the cost of zero-knowledge privacy enforcement. Verification operations, including signature verification and zkSNARK proof validation, complete in 96.3 ms, with the EdDSA signature verification itself taking only 1.8 ms. These results confirm the system's efficiency in both proof generation and verification scenarios.

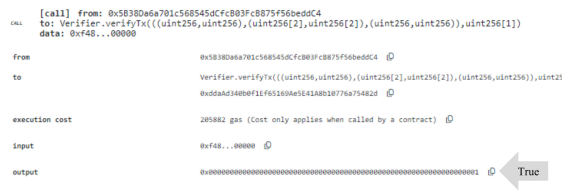

Fig. 3. Verification of Proof.

TABLE IV
GAS COST ANALYSIS ON ETHEREUM VIRTUAL TESTNET (REMIX IDE)

Operation	Transaction Hash	Gas Used
Deploy Verifier Contract	0x8a3d...c42f	100,000
Verify zkSNARK Proof	0x5b21...d87a	20,000
Store Merkle Root	0x3c9e...f12b	8,000
Update Issuer Registry	0x7d84...a65c	6,000

The Table IV provides a summary of the gas costs for various operations in the proposed eKYC system, measured on the Ethereum Virtual Testnet using Remix IDE. The on-chain resource use related to important processes, such deploying the verifier contract, confirming zkSNARK proofs, keeping the Merkle root, and updating the issuer registry, is reflected in these gas costs. Since the virtual testnet is intended for testing and development, its gas consumption is substantially lower than that of live networks. For example, it costs 100,000 gas to deploy the verifier contract, while it costs 20,000 gas to verify the zkSNARK proof. The computational efficiency of updating the issuer registry and storing the Merkle root is demonstrated by the fact that they each require 8,000 and 6,000 gas, respectively. All things considered, the outcomes show that the eKYC system is set up for minimal gas consumption in a testnet setting, guaranteeing effective use of blockchain resources.

TABLE V
SECURITY ANALYSIS TABLE

Threat Type	Mitigation Mechanism	Effectiveness
Replay Attack	Unique nonce per proof	High
Collusion Attack	Selective disclosure via Merkle paths.	Medium-High
DoS Attack	Efficient verification & lightweight proofs.	High
Data Tampering	Immutable Merkle root on blockchain.	High

The security features built into the suggested eKYC system to counter particular attacks are described in the Table V. Implementing a distinct nonce for every proof ensures non-reusability and reduces replay assaults. Selective disclosure utilizing Merkle routes reduces the danger of collusion by avoiding the pooling of disparate bits of information from various verifications. The system uses optimized proof sizes and effective proof verification procedures to counteract DoS attacks while lowering computing cost. By keeping the Merkle root on the blockchain in an unchangeable format, data integrity is preserved and any unwanted changes to the credential data are avoided. The security foundation of the eKYC system is made more resilient overall by these integrated solutions.

VI. CONCLUSION

In this article, we address privacy concerns by allowing users to share only the necessary data, ensuring secure and private verification. The system also minimizes the risks of centralized data storage by decentralizing the storage of sensitive information, reducing the potential for unauthorized access and improving overall security. Additionally, using zero-knowledge proofs and Merkle trees means that user data is never fully exposed during the verification process. With blockchain technology, it guarantee the immutability of the data, adding an extra layer of trust for users and organizations alike. Ultimately, this system provides a reliable and scalable solution for secure identity verification in today's digital world, meeting privacy regulations and offering peace of mind for users.

There were some technical hurdles during implementation. The integration of ZKPs into existing systems proved challenging, especially when handling cryptographic tasks on low-power devices. Scaling the system to handle large numbers of users and verifications without losing performance is another area that needs work. Additionally, the reliance on zk-SNARKs' trusted setup introduces some complexity and potential risk. In order to further enhance user privacy, we intend to concentrate on creating the full functional user wallet, enhancing scalability, and investigating zero-knowledge verified credentials.

ACKNOWLEDGMENT

This work was supported by JST SPRING, Grant Number JPMJSP2139.

REFERENCES

[1] A. Gomaa, O. Rashed, A. Refaey, A. Mohamed, M. Sayed, and M. Rashwan, "A new framework for an eKYC system," in *Proceedings of the 20th Conference on Language Engineering (ESOLEC)*, 2022, pp. 11–18.

[2] S. Bhatia, L. Vishwakarma, and D. Das, "Cost-efficient Blockchain-based e-KYC Platform using Biometric verification," in *International Conference on Information Networking*, 2024, pp. 403–408.

[3] P. Yadav and R. Chandak, "Transforming the Know Your Customer (KYC) Process using Blockchain," in *2019 6th IEEE International Conference on Advances in Computing, Communication and Control (ICAC3)*, Dec. 2019.

[4] O. Goldreich and Y. Oren, "Definitions and properties of zero-knowledge proof systems," *Journal of Cryptology*, vol. 7, no. 1, pp. 1–32, Dec. 1994.

[5] U. Feige, A. Fiat, and A. Shamir, "Zero Knowledge Proofs of Identity," in *Annual ACM Symposium on Theory of Computing*, 1987, pp. 210–217.

[6] T. Chen, H. Lu, T. Kunpittaya, and A. Luo, "A Review of zk-SNARKs," https://arxiv.org/abs/2202.06877v4, Feb. 2022, accessed: Sep. 03, 2024.

[7] A. M. Pinto, "An Introduction to the Use of zk-SNARKs in Blockchains," in *Springer Proceedings in Business and Economics*, 2020, pp. 233–249.

[8] O. Amine, K. Baghery, Z. Pindado, and C. Ràfols, "Simulation extractable versions of Groth's zk-SNARK revisited," *International Journal of Information Security*, vol. 23, no. 1, pp. 431–445, Feb. 2024.

[9] H. Lipmaa, "Polymath: Groth16 Is Not the Limit," in *Lecture Notes in Computer Science*, 2024, pp. 170–206.

[10] E. Heeren-Moon, "Risk, reputation and responsibility: Cybersecurity and centralized data in United States civilian federal agencies," *Telecommunications Policy*, vol. 47, no. 2, p. 102502, Mar. 2023.

[11] "General Data Protection Regulation (GDPR) – Legal Text," https://gdpr-info.eu/, 2024, accessed: Sep. 03, 2024.

[12] "Self-Sovereign Identity - Alex Preukschat, Drummond Reed - Google Books," https://books.google.co.jp/books?hl=en&lr=&id=BfQ1EAAAQBAJ&oi=fnd&pg=PA1&dq=Self-Sovereign+Identity+(SSI), accessed: Sep. 03, 2024.

[13] S. Cucko, S. Becirovic, A. Kamisalic, S. Mrdovic, and M. Turkanovic, "Towards the Classification of Self-Sovereign Identity Properties," *IEEE Access*, vol. 10, pp. 88 306–88 329, 2022.

[14] K. S. Chandraprabha, "Blockchain-Based Implementation on Electronic Know Your Customer (e-KYC)," in *Lecture Notes on Data Engineering and Communications Technologies*, 2023, vol. 171, pp. 281–297.

[15] B. Jaber, D. Kriwiesh, and M. A. Alragheb, "A Blockchain Framework in the Banking Sector Based in e-KYC System Conceptual Framework," in *2nd International Conference on Cyber Resilience (ICCR)*, 2024.

[16] S. Fugkeaw, "Enabling Trust and Privacy-Preserving e-KYC System Using Blockchain," *IEEE Access*, vol. 10, pp. 49 028–49 039, 2022.

[17] M. A. Hannan, M. A. Shahriar, M. S. Ferdous, M. J. M. Chowdhury, and M. S. Rahman, "A systematic literature review of blockchain-based e-KYC systems," *Computing*, vol. 105, no. 10, pp. 2089–2118, Oct. 2023.

[18] V. Schlatt, J. Sedlmeir, S. Feulner, and N. Urbach, "Designing a Framework for Digital KYC Processes Built on Blockchain-Based Self-Sovereign Identity," *Information & Management*, vol. 59, no. 7, p. 103553, Nov. 2022.

[19] W. Li, C. Meese, H. Guo, and M. Nejad, "Blockchain-Enabled Identity Verification for Safe Ridesharing Leveraging Zero-Knowledge Proof," in *2020 3rd International Conference on Hot Information-Centric Networking (HotICN)*, Dec. 2020, pp. 18–24.

[20] B. N. Eddine, A. Ouaddah, and A. Mezrioui, "Exploring blockchain-based Self Sovereign Identity Systems: Challenges and comparative analysis," in *2021 3rd Conference on Blockchain Research and Applications for Innovative Networks and Services (BRAINS)*, Sep. 2021, pp. 21–22.

[21] I. Ahmed, "Corresponding GitHub repository of the proposed solution," https://github.com/istiaque010/Enhanching_Privacy_in_eKYC/, 2024, accessed: Aug. 03, 2024.